Speech To Text Explanation

This note explains, in a simple beginner-friendly way, how your current speech-to-text feature works in this project.

1. What is being used?

Your project is now using:

- React on the frontend
- FastAPI on the Python backend
- Faster Whisper as the speech-to-text model
- MediaRecorder in the browser to capture your microphone audio

So the actual speech recognition is being done by the Python model, not by the browser.

2. Simple flow

The full flow is:

1. You click the `Mic` button in the interview page.
2. The browser records your voice using the microphone.
3. When you stop recording, the frontend sends the audio file to the Python API.
4. The Python API passes that audio to the Faster Whisper model.
5. Faster Whisper listens to the audio and converts spoken words into text.
6. The backend sends that text back to React.
7. React fills the answer box with the transcribed text.

3. What happens in the frontend?

Frontend file:

- `src/app/interviews/[sessionId]/page.tsx`

What it does:

• asks for microphone permission

• records audio using `MediaRecorder`

• stores the recorded audio chunks

• creates one audio blob

• sends that audio to the backend using the helper in:

- `src/lib/transcription-client.ts`

So React is not doing the actual speech recognition. It is only recording and sending audio.

4. What happens in the backend?

Backend files:

• `ai_service/app/main.py`

• `ai_service/app/transcription.py`

`main.py`

This file creates the API endpoint:

• `POST /transcribe-audio`

This endpoint receives the recorded audio from the frontend.

`transcription.py`

This file loads the Faster Whisper model and uses it to convert audio into text.

Right now the default model is:

• `medium.en`

That means:

• `medium` = a stronger and more accurate Whisper model than smaller ones

• `.en` = optimized for English speech

5. Why Faster Whisper is used

Faster Whisper is a speech-to-text library based on OpenAI Whisper models.

It is used because:

• it works in Python

• it is usually more accurate than many simple browser-only options

• it supports English speech well

• it can run locally on your machine

6. Why transcription can take time

It may feel slow because:

• audio must first be recorded

• then uploaded to the backend

• then processed by the Whisper model

• then the text is sent back to the frontend

Also, bigger models like `medium.en` are more accurate, but slower than smaller models like `base.en` or `small.en`.

So there is always a tradeoff:

• smaller model = faster, but less accurate

• bigger model = slower, but more accurate

7. Why errors can still happen

Even good speech-to-text models can make mistakes if:

• the voice is very soft

• the mic is far away

• background noise exists

• pronunciation is fast or unclear

• technical terms are uncommon

So if you whisper a lot, the model may still misunderstand some words.

8. In one line

Your app records your voice in React, sends the audio to the Python FastAPI backend, and the Faster Whisper model converts that audio into text.

9. File summary

• `src/app/interviews/[sessionId]/page.tsx`

records microphone audio

• `src/lib/transcription-client.ts`

sends recorded audio to the backend

• `ai_service/app/main.py`

exposes the transcription API endpoint

• `ai_service/app/transcription.py`

loads and runs the Faster Whisper model

10. Beginner-friendly short example

Think of it like this:

• React = recorder

• FastAPI = messenger

• Faster Whisper = listener and writer

You speak -> React records -> Python receives -> Whisper listens -> text comes back.